Deep Residual Networks

A Complete Visual and Mathematical Exploration

Chandra Prakash Bathula

November 2025

Covering: Skip Connections · Residual Blocks · Architecture Variants · Element-Wise Addition · Gradient Flow · Training & Initialization · Advanced Blocks · Theoretical Foundations · Implementation · Applications

Contents

1 Introduction to Deep Residual Networks **1**
1.1 From Shallow to Deep: The Road to Residual Learning . . . 1
1.2 The Three Root Causes of Degradation . . . 1
1.3 The Residual Learning Hypothesis . . . 2
1.4 Historical Impact at a Glance . . . 2

2 Skip Connections and Residual Learning **3**
2.1 Motivation and Mathematical Foundation . . . 3
2.2 What Is a Skip Connection? . . . 3
2.3 Gradient Flow Through Skip Connections . . . 4
2.4 Three Types of Skip Connections . . . 4
2.5 Effect on the Optimization Landscape . . . 4
2.6 ResNets as Ensembles of Shallow Networks . . . 5

3 Residual Blocks: All Variants **6**
3.1 Basic Residual Block (ResNet-18/34) . . . 6
3.2 Bottleneck Residual Block (ResNet-50/101/152) . . . 6
3.3 Pre-Activation Residual Block (ResNet v2) . . . 7
3.4 Wide Residual Block (WRN) . . . 7
3.5 ResNeXt Block (Grouped Convolutions) . . . 8
3.6 SE-ResNet: Squeeze-and-Excitation Block . . . 8

4 Full ResNet Architectures: Stage-by-Stage **12**
4.1 The Stem Layer . . . 12
4.2 Stage-by-Stage Architecture Tables . . . 13
4.3 Channel and Spatial Flow Visualization . . . 13
4.4 Transition Blocks: Handling Shape Changes Between Stages . . . 13

5 Element-Wise Addition: Vectors and Feature Maps **16**
5.1 Overview . . . 16
5.2 Element-Wise Addition of Vectors (FC/MLP Residuals) . . . 16
5.2.1 Formal Definition . . . 16
5.2.2 Visual Walkthrough: 5-D Vector Addition . . . 16
5.2.3 Full FC Residual Block . . . 17
5.3 Element-Wise Addition of Feature Maps (CNN Residuals) . . . 17
5.3.1 Feature Maps as 3-D Tensors . . . 17
5.3.2 Visual Walkthrough: Feature Map Addition . . . 18
5.4 Identity vs Projection Shortcut . . . 18
5.5 The Role of ReLU After Addition . . . 18
5.6 Summary: Vector vs Feature-Map Addition . . . 18

6 Weight Initialization and Normalization **20**
6.1 Kaiming / He Initialization . . . 20
6.1.1 The Problem with Naive Initialization . . . 20
6.1.2 He Initialization (for ReLU) . . . 20
6.1.3 Zero-Init of Final BN Scale . . . 20
6.2 Batch Normalization . . . 21
6.2.1 Motivation . . . 21
6.2.2 BN Position: Before or After Addition? . . . 21
6.2.3 Alternatives to BatchNorm . . . 21

7 Training ResNets: Schedules, Regularization, and Tricks **23**
7.1 Learning Rate Schedules . . . 23
7.1.1 Step Decay . . . 23
7.1.2 Cosine Annealing . . . 23
7.1.3 Linear Warmup . . . 23
7.2 Stochastic Depth (DropPath) . . . 23
7.3 Data Augmentation Strategies . . . 24
7.4 Complete Training Recipe (ResNet-50 on ImageNet) . . . 25

8 PyTorch Implementation **26**
8.1 Basic Residual Block . . . 26
8.2 Bottleneck Block . . . 27
8.3 Full ResNet Class . . . 27
8.4 Transfer Learning with ResNet . . . 28
8.5 Common Failure Modes and Debugging . . . 29

9 Theoretical Foundations **30**
9.1 ResNets as Discretized ODEs . . . 30
9.1.1 Neural ODEs (Chen et al., 2018) . . . 30
9.2 Ensemble Interpretation . . . 30
9.3 Loss Landscape Analysis . . . 31
9.4 Gradient Flow Analysis . . . 31

10 Modern Architecture Variants **33**
10.1 DenseNet: All-to-All Connections . . . 33
10.2 ConvNeXt: Modernizing ResNets (2022) . . . 33
10.3 Vision Transformers (ViT) and Residuals . . . 33
10.4 Architecture Family Comparison . . . 33

11 Applications and Transfer Learning **36**
11.1 Computer Vision Tasks . . . 36
11.1.1 Image Classification . . . 36
11.1.2 Object Detection: Faster R-CNN and YOLO . . . 36
11.1.3 Semantic Segmentation: DeepLab and U-Net . . . 36
11.2 Medical Imaging . . . 36
11.3 Beyond Vision . . . 37

12 Limitations and Open Research Directions **38**
12.1 Known Limitations . . . 38
12.2 Active Research Directions . . . 38
12.2.1 Dynamic Residual Paths . . . 38
12.2.2 Neural Architecture Search (NAS) . . . 38

12.2.3 Zero-Cost Proxies 38
12.2.4 Mechanistic Interpretability 38

References **40**

Chapter 1

Introduction to Deep Residual Networks

1.1 From Shallow to Deep: The Road to Residual Learning

The journey of convolutional neural networks (CNNs) has been a story of going deeper. Early networks like **LeNet-5** (1998) contained only a few layers and were sufficient for digit recognition. A decade later, **AlexNet** (2012) demonstrated that depth, combined with GPUs and ReLU activations, could scale performance to the ImageNet challenge. **VGG-16/19** (2014) extended this idea—stacking 3×3 convolutions to reach 19 layers—but training deeper models soon hit a wall: beyond a certain depth, both training and test errors increased. This "**degradation problem**" puzzled researchers; deeper networks should, in principle, perform at least as well as their shallower counterparts.

Figure 1.1: Milestones in CNN evolution. Each leap involved greater depth until ResNets overcame the degradation barrier.

1.2 The Three Root Causes of Degradation

1. Vanishing Gradients

When many nonlinear layers are stacked, gradient magnitudes shrink exponentially during backpropagation. For a network with L layers and per-layer Jacobian $J_l = \partial x_{l+1}/\partial x_l$:

$$\frac{\partial \mathcal{L}}{\partial x_1} = \frac{\partial \mathcal{L}}{\partial x_L} \prod_{l=1}^{L-1} J_l.$$

If $\|J_l\| < 1$ on average, the product decays exponentially and early layers receive negligible gradient signal.

2. Degradation Problem (Not Overfitting)

Even with BatchNorm and ReLU, a 56-layer plain CNN achieves *higher training error* than a 20-layer model on CIFAR-10. This is not overfitting—the test error is also worse. The optimizer simply fails to find good solutions in the high-dimensional, poorly conditioned loss landscape.

3. Optimization Barriers

The loss surface of very deep plain networks is riddled with saddle points and flat plateaus. SGD requires many steps to escape, and the effective learning signal diminishes along the way.

1.3 The Residual Learning Hypothesis

Proposed by **Kaiming He et al.** in 2015, the key insight is:

Core Hypothesis

Instead of expecting a stack of layers to learn an arbitrary mapping $\mathcal{H}(x)$, let them learn the *residual* $\mathcal{F}(x) = \mathcal{H}(x) - x$. The original mapping becomes $\mathcal{H}(x) = \mathcal{F}(x) + x$.

If the identity is optimal, the network just drives $\mathcal{F}(x) \to 0$—a much easier optimization target than relearning the identity from scratch.

1.4 Historical Impact at a Glance

Model	Year	Depth	ImageNet Top-5 Error
AlexNet	2012	8	16.4%
VGG-16	2014	16	7.3%
GoogLeNet	2014	22	6.7%
ResNet-34	2015	34	5.7%
ResNet-152	2015	152	**4.49%**
ResNet ensemble	2015	–	**3.57%**

Table 1.1: ImageNet ILSVRC top-5 error progression. ResNet-152 halved the error of VGG.

Chapter 2

Skip Connections and Residual Learning

2.1 Motivation and Mathematical Foundation

Let $\mathcal{H}(x)$ be the desired mapping. ResNets reparameterize it as:

$$\mathcal{F}(x) = \mathcal{H}(x) - x \quad \Longrightarrow \quad \mathcal{H}(x) = \mathcal{F}(x) + x.$$

This reparameterization has two key advantages:

- If the identity is optimal, $\mathcal{F}(x) \to 0$ is trivially achievable (push weights toward zero).
- The gradient of the loss always has a direct, unimpeded path through the $+x$ term.

2.2 What Is a Skip Connection?

A **skip connection** (shortcut) is a direct link bypassing one or more layers, summing its output with the main-path output:

$$y = \mathcal{F}(x, \{W_i\}) + x.$$

The "+" is an element-wise addition — no learnable parameters, no nonlinearity.

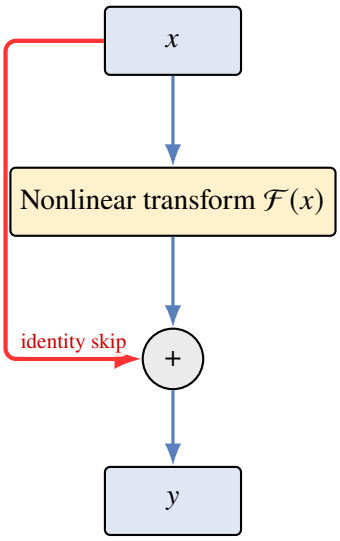


Figure 2.1: Skip connection: the input x bypasses the nonlinear transform and is added at the output.

2.3 Gradient Flow Through Skip Connections

The backward pass through the addition node:

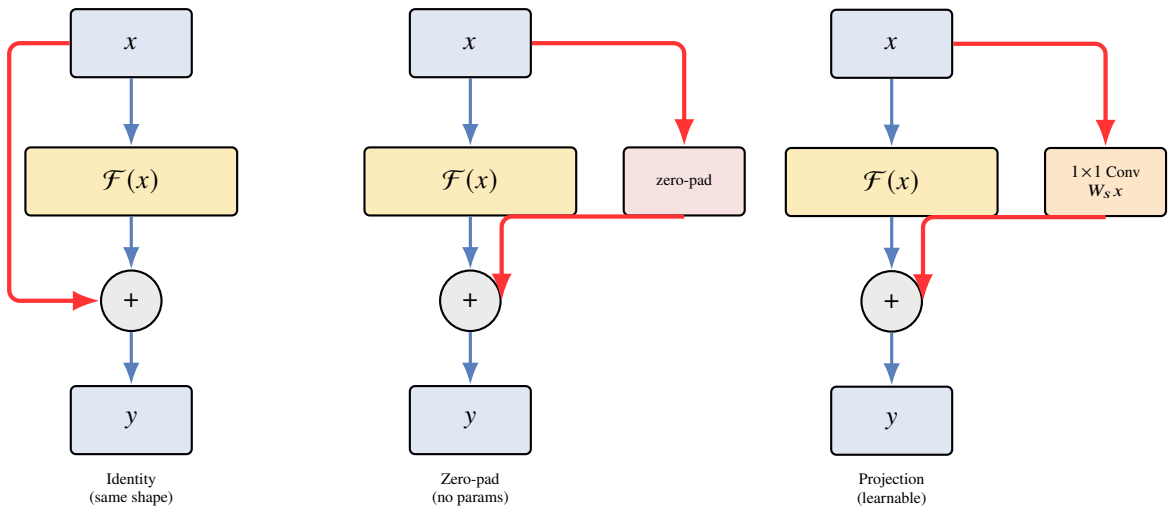
$$\frac{\partial \mathcal{L}}{\partial x} = \frac{\partial \mathcal{L}}{\partial y} \cdot \left(I + \frac{\partial \mathcal{F}}{\partial x} \right).$$

The identity term I guarantees a non-zero gradient path regardless of $\partial \mathcal{F} / \partial x$. Extending this across L blocks:

$$\frac{\partial \mathcal{L}}{\partial x_0} = \frac{\partial \mathcal{L}}{\partial x_L} \prod_{l=0}^{L-1} \left(I + \frac{\partial \mathcal{F}_l}{\partial x_l} \right) = \frac{\partial \mathcal{L}}{\partial x_L} \left[I + \sum_l \frac{\partial \mathcal{F}_l}{\partial x_l} + \cdots \right].$$

The expansion shows that gradient always contains a direct term $\partial \mathcal{L} / \partial x_L$, preventing complete vanishing.

2.4 Three Types of Skip Connections



Option A: Identity Option B: Zero-Pad Option C: Projection

Figure 2.2: Three shortcut options from the original ResNet paper. Option A (identity) is used when shapes match. Option B (zero-padding) adds zeros to match channels with no extra parameters. Option C (1×1 convolution) is the most expressive and is used when spatial size or channel count changes.

Option	When used	Parameters added	Performance
A: Identity	Same C, H, W	Zero	Best (no overhead)
B: Zero-pad	Channel increase only	Zero	Slightly lower
C: Projection	Channel or spatial change	$C_{out} \times C_{in} \times 1 \times 1$	Best when needed

Table 2.1: Comparison of shortcut options. The ResNet paper found Option C adds marginal cost but works best.

2.5 Effect on the Optimization Landscape

ResNets dramatically smooth the loss surface. Without residuals, deep nets exhibit chaotic, high-curvature landscapes. With them, the surface resembles a gentle valley.

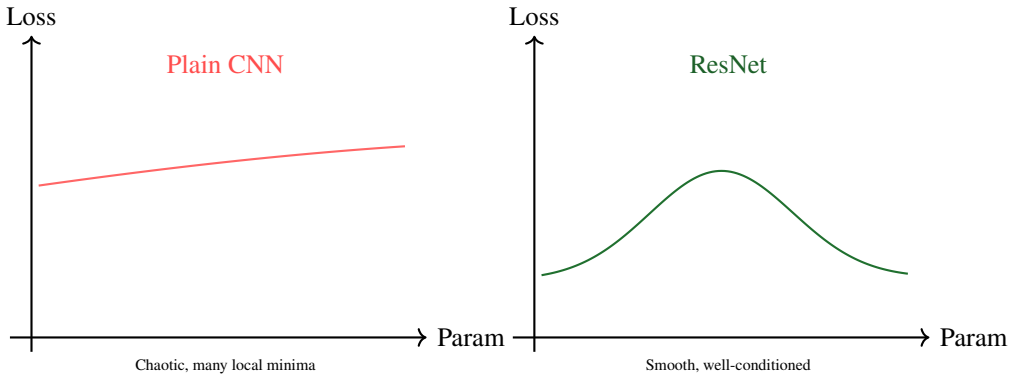


Figure 2.3: Conceptual loss landscape comparison. ResNets produce smoother, more convex-like surfaces that SGD can navigate efficiently.

2.6 ResNets as Ensembles of Shallow Networks

A powerful interpretation (Veit et al., 2016): because each block can be bypassed, a ResNet with L blocks implicitly represents 2^L paths of varying lengths. The gradient effectively averages over an exponential number of shallow networks.

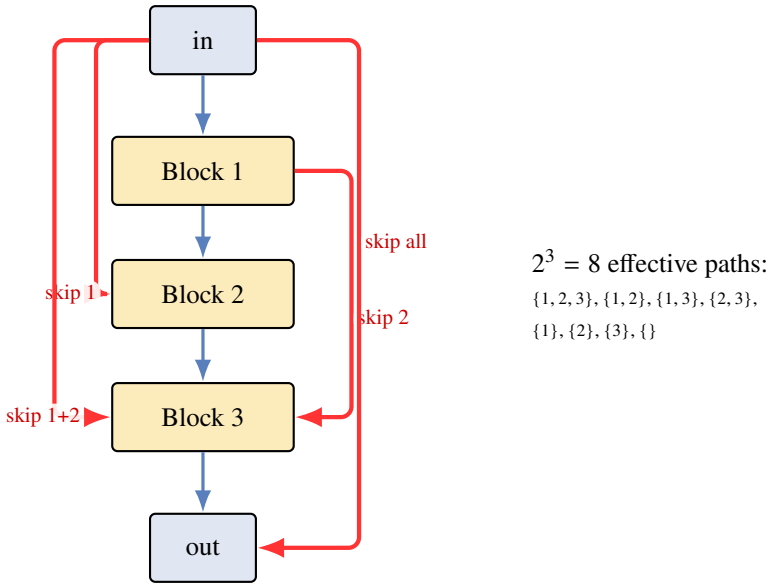


Figure 2.4: A 3-block ResNet implicitly defines $2^3 = 8$ sub-network paths. At inference, predictions are effectively an ensemble over all paths. Removing any one block gracefully degrades rather than catastrophically fails.

Chapter 3

Residual Blocks: All Variants

3.1 Basic Residual Block (ResNet-18/34)

Structure and Mathematics

Two 3×3 convolutions with BatchNorm and ReLU. The input bypasses both layers:

$$z_1 = \mathrm{ReLU}(\mathrm{BN}(W_1 * x)), \quad z_2 = \mathrm{BN}(W_2 * z_1), \quad y = \mathrm{ReLU}(z_2 + x).$$

Parameter Count

For input/output channels C and kernel size 3×3:

$$\text{Params} = 2 \times (3 \times 3 \times C \times C) + 2 \times C = 18C^2 + 2C \approx 18C^2.$$

3.2 Bottleneck Residual Block (ResNet-50/101/152)

Motivation and Structure

For deep networks, the basic block's $18C^2$ parameters become prohibitive. The bottleneck reduces inner channels by factor 4:

$$1 \times 1 \xrightarrow{\text{reduce}} 3 \times 3 \xrightarrow{\text{process}} 1 \times 1 \xrightarrow{\text{expand}}$$

$$y = x + W_3 * \sigma(\mathrm{BN}(W_2 * \sigma(\mathrm{BN}(W_1 * x)))).$$

Parameter Comparison: Basic vs Bottleneck

Block	**Layers**	**Formula**	**Params (C=64)**
Basic	3×3, 3×3	$18C^2$	73,728
Bottleneck	1×1, 3×3, 1×1	$C^2 + 9C^2 + 4C^2 = 14C^2$ ($\times$4 outer)	69,632

Table 3.1: Bottleneck achieves similar parameter count with greater depth and expressiveness.

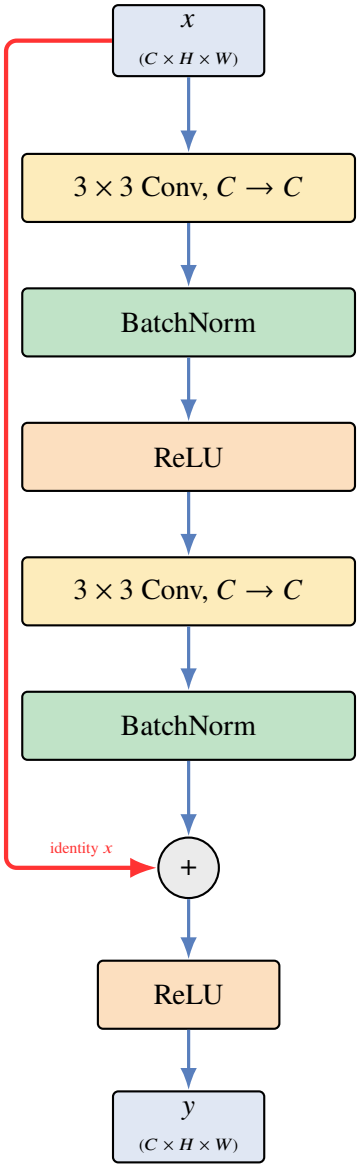


Figure 3.1: Basic residual block used in ResNet-18 and ResNet-34. The identity shortcut requires $\mathcal{F}(x)$ to have the same spatial dimensions and channel count as x.

3.3 Pre-Activation Residual Block (ResNet v2)

Key Insight

He et al. (2016) observed that placing BN and ReLU *before* the convolutions keeps the identity path **completely clean**—no activation ever modifies the shortcut signal:

$$y = x + W_2 * \sigma(\mathrm{BN}(W_1 * \sigma(\mathrm{BN}(x)))).$$

3.4 Wide Residual Block (WRN)

Wide ResNets (Zagoruyko & Komodakis, 2016) increase channel *width* rather than depth. A widening factor k multiplies the channel count:

$$\text{WRN-}d\text{-}k: \quad C \to kC \quad \text{in each block.}$$

WRN-28-10 (28 layers, $k = 10$) outperforms ResNet-1001 on CIFAR while training 8× faster.

3.5 ResNeXt Block (Grouped Convolutions)

ResNeXt (Xie et al., 2017) introduces **cardinality** g as a new dimension of capacity: instead of one 3×3 conv, use g parallel groups:

$$y = x + \sum_{i=1}^{g} \mathcal{T}_i(x),$$

where each $\mathcal{T}_i$ processes C/g channels independently.

3.6 SE-ResNet: Squeeze-and-Excitation Block

Hu et al. (2018) add a **channel attention** mechanism: the network learns to recalibrate channel responses by globally summarizing each channel and then applying learned channel-wise scales.

SE Module Mathematics

1. **Squeeze:** Global average pool each channel to a scalar: $s_c = \frac{1}{H \times W} \sum_{h,w} X_{c,h,w}$, giving $\mathbf{s} \in \mathbb{R}^C$.
2. **Excitation:** Two FC layers with bottleneck ratio r: $\mathbf{e} = \sigma(W_2\, \delta(W_1\, \mathbf{s}))$, where δ = ReLU, σ = Sigmoid.
3. **Scale:** Multiply each channel: $\tilde{X}_{c,h,w} = e_c \cdot X_{c,h,w}$.

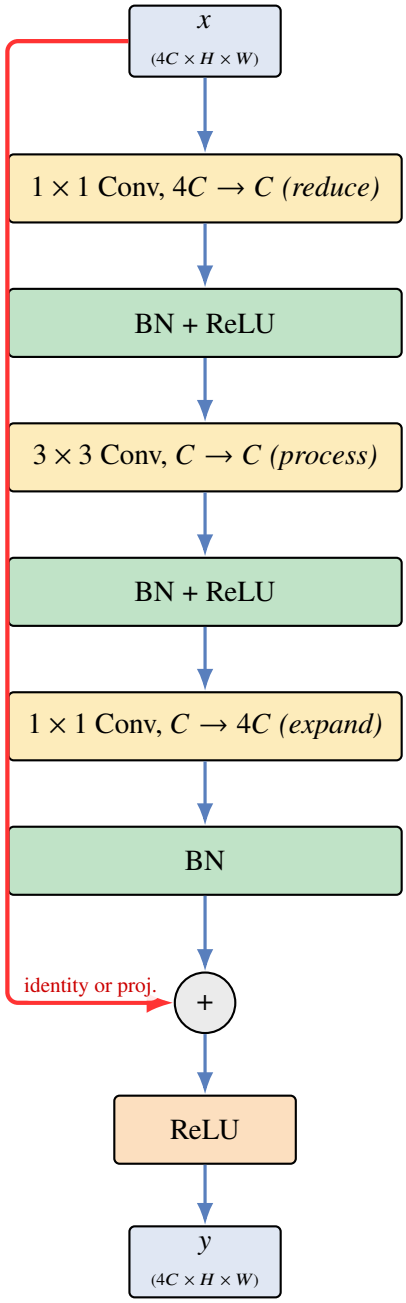
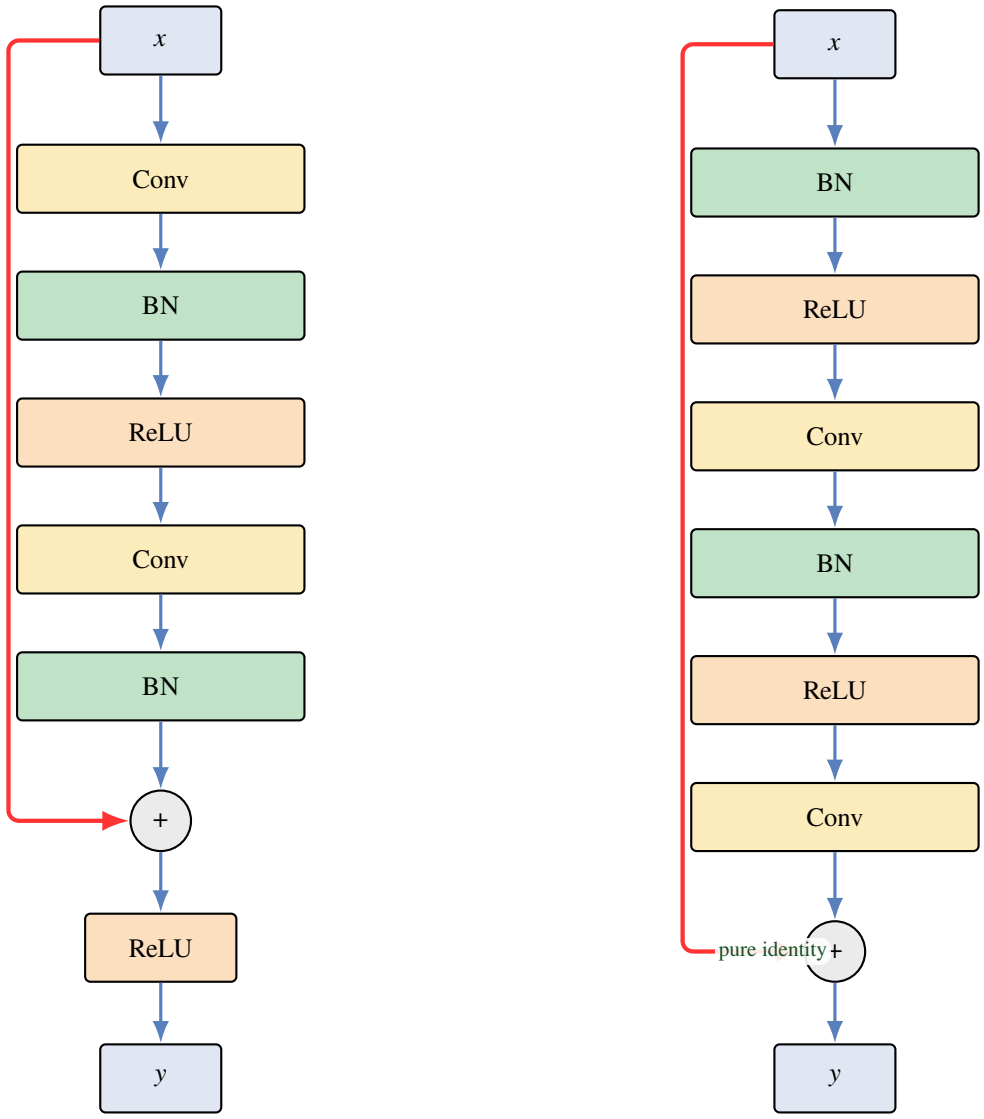


Figure 3.2: Bottleneck block. The inner dimension is C (typically 64) while the outer dimension is $4C$ (256). This reduces FLOPs from $O(C^2)$ to $O(C^2/4 + C^2)$, saving roughly 75% of computation vs a basic block at same outer dimension.



(a) Post-activation (v1)
ReLU modifies skip path

(b) Pre-activation (v2)
Pure identity skip path

Figure 3.3: Post-activation (ResNet v1) vs Pre-activation (ResNet v2). The pre-activation form allows gradients to flow through the identity without any nonlinear distortion, enabling stable training of 1000+ layer networks.

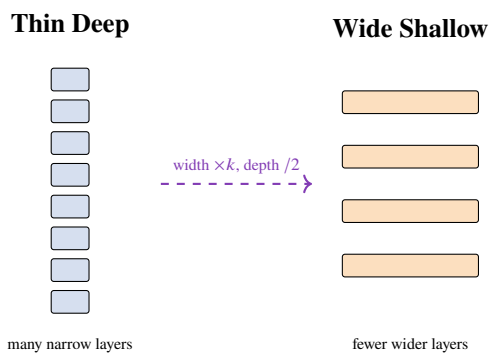


Figure 3.4: Wide ResNets trade depth for width. Wider layers have more representational capacity per layer and parallelize better on modern GPUs.

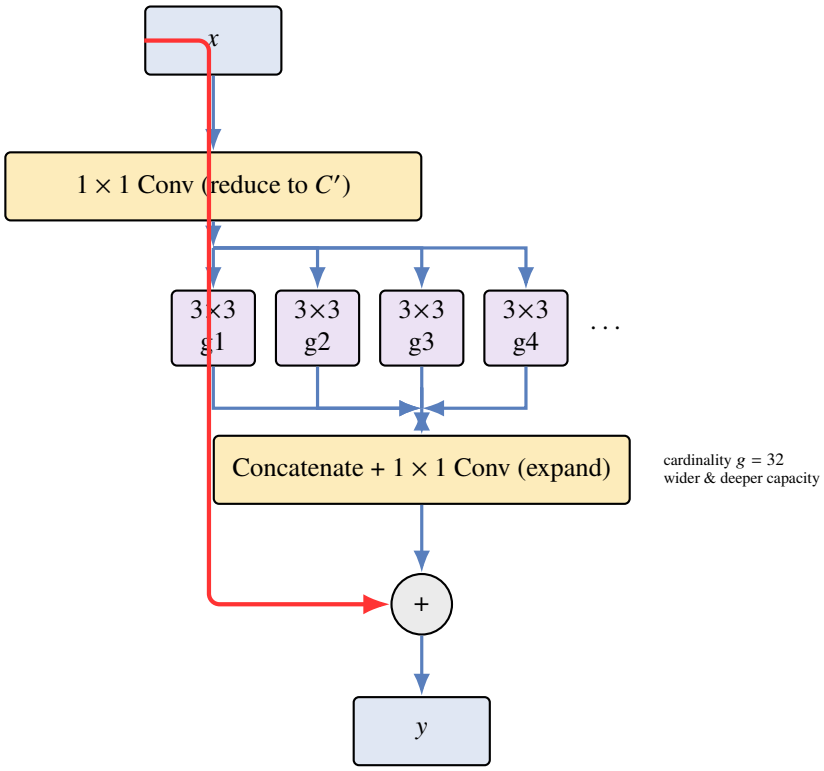


Figure 3.5: ResNeXt block with grouped convolutions. Cardinality g controls the number of parallel transformation paths. ResNeXt-50 with $g = 32$ outperforms ResNet-101 with similar FLOP budget.

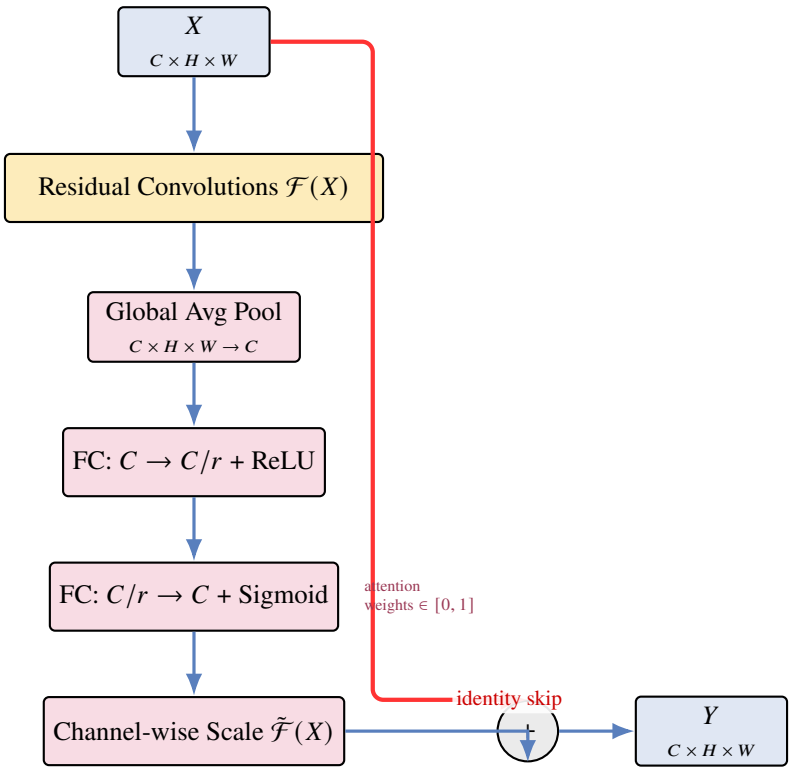


Figure 3.6: SE-ResNet block. The Squeeze-and-Excitation branch learns which channels are most informative and rescales them adaptively. The residual skip still connects X directly to the addition node.

Chapter 4

Full ResNet Architectures: Stage-by-Stage

4.1 The Stem Layer

Every ResNet begins with a **stem**: a large 7×7 conv with stride 2 followed by 3×3 max-pooling with stride 2. This aggressively reduces spatial size from 224×224 to 56×56 (a factor of 16×) before any residual blocks, reducing FLOPs substantially while preserving early edge features.

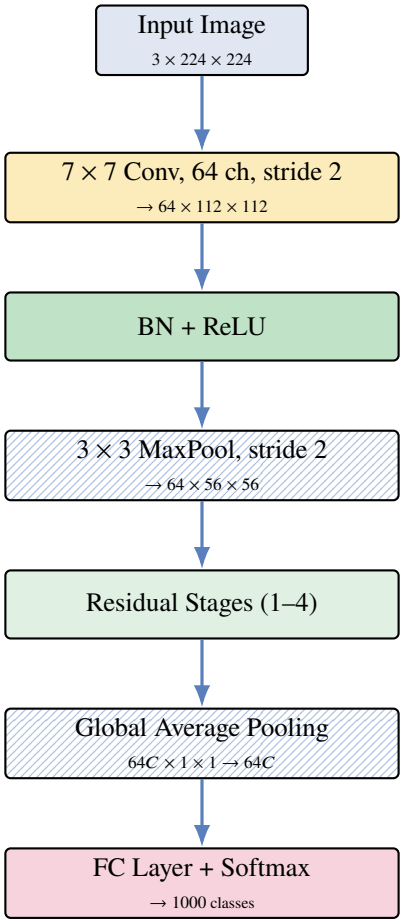


Figure 4.1: Complete ResNet pipeline from raw image to class prediction. The stem downsamples aggressively before residual processing begins. Global Average Pooling replaces the large FC layers used in AlexNet/VGG, drastically reducing parameters.

Why Global Average Pooling?

AlexNet used three fully-connected layers totalling ~60M parameters. Global Average Pooling (GAP) collapses each feature map to a single value, producing a vector of length C regardless of input size. This:

- Reduces parameters from $7 \times 7 \times 512 \times 4096 \approx 100$M to *zero* (no weights).
- Makes the network *fully convolutional*—it can process any input size.
- Acts as structural regularization, reducing overfitting.

4.2 Stage-by-Stage Architecture Tables

ResNet-18 and ResNet-34 (Basic Blocks)

Stage	Output size	ResNet-18	ResNet-34	Channels
Stem	56×56	7×7 conv, MP	7×7 conv, MP	64
Stage 1	56×56	$[3 \times 3, 3 \times 3] \times 2$	$[3 \times 3, 3 \times 3] \times 3$	64
Stage 2	28×28	$[3 \times 3, 3 \times 3] \times 2$	$[3 \times 3, 3 \times 3] \times 4$	128
Stage 3	14×14	$[3 \times 3, 3 \times 3] \times 2$	$[3 \times 3, 3 \times 3] \times 6$	256
Stage 4	7×7	$[3 \times 3, 3 \times 3] \times 2$	$[3 \times 3, 3 \times 3] \times 3$	512
Head	1×1	GAP + FC-1000	GAP + FC-1000	–
Total params		11.7M	21.8M	

Table 4.1: ResNet-18 and ResNet-34 architecture. Bracket notation $[a, b] \times n$ means n basic blocks each containing layers a and b. Spatial resolution halves and channels double between stages.

ResNet-50, 101, and 152 (Bottleneck Blocks)

Stage	Size	ResNet-50	ResNet-101	ResNet-152	Channels
Stem	56×56	7×7 conv, 64ch, s2; 3×3 MaxPool, s2			64
Stage 1	56×56	$[1, 3, 1] \times 3$	$[1, 3, 1] \times 3$	$[1, 3, 1] \times 3$	256
Stage 2	28×28	$[1, 3, 1] \times 4$	$[1, 3, 1] \times 4$	$[1, 3, 1] \times 8$	512
Stage 3	14×14	$[1, 3, 1] \times 6$	$[1, 3, 1] \times 23$	$[1, 3, 1] \times 36$	1024
Stage 4	7×7	$[1, 3, 1] \times 3$	$[1, 3, 1] \times 3$	$[1, 3, 1] \times 3$	2048
Head	1×1	GAP+FC	GAP+FC	GAP+FC	–
Params		25.6M	44.5M	60.2M	
Top-1 Acc		76.1%	77.4%	78.3%	

Table 4.2: Bottleneck-based ResNets. $[1, 3, 1]$ denotes the 1×1-3×3-1×1 bottleneck. The bulk of parameters are in Stage 3, which is expanded most aggressively.

4.3 Channel and Spatial Flow Visualization

4.4 Transition Blocks: Handling Shape Changes Between Stages

At the boundary between stages, spatial resolution halves and channels double. This requires a **projection shortcut** (1×1 conv, stride 2):

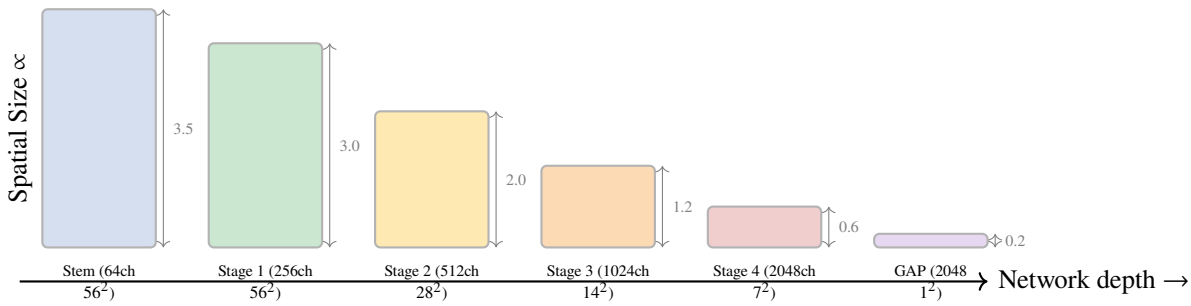


Figure 4.2: Spatial resolution (bar height) and channel count progression through a ResNet-50. As depth increases, spatial area shrinks while channel depth grows, maintaining roughly constant information capacity.

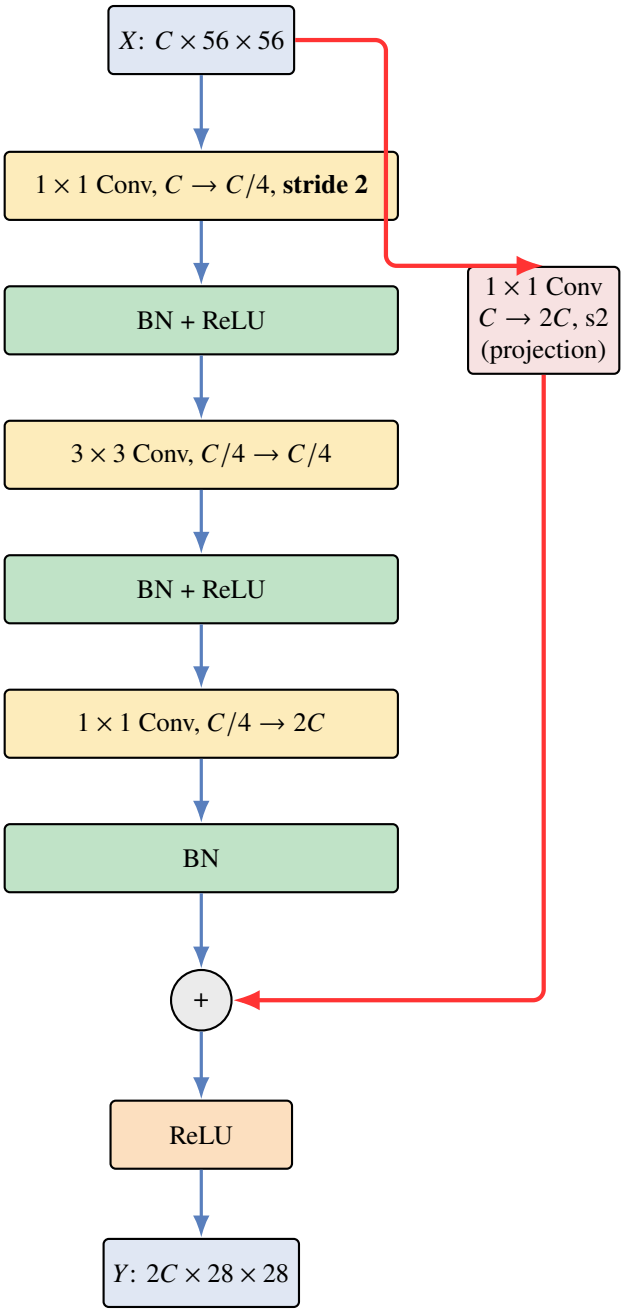


Figure 4.3: Transition (downsampling) block at stage boundaries. The main path uses stride-2 in the first 1×1 conv. The projection shortcut matches with its own stride-2 1×1 conv. Both paths produce tensors of shape $2C \times 28 \times 28$ before element-wise addition.

Chapter 5

Element-Wise Addition: Vectors and Feature Maps

5.1 Overview

At the core of every residual block is one arithmetic operation: **element-wise addition**. Two tensors of identical shape are added scalar-by-scalar—no mixing across positions. This chapter dissects that operation in both the vector (FC/MLP) and 3-D feature-map (CNN) settings.

5.2 Element-Wise Addition of Vectors (FC/MLP Residuals)

5.2.1 Formal Definition

Let $\mathbf{x} \in \mathbb{R}^d$ be an input vector and $\mathcal{F}(\mathbf{x}) \in \mathbb{R}^d$ the transformed vector. The residual sum is:

$$y_i = x_i + \mathcal{F}(\mathbf{x})_i, \quad i = 1, \ldots, d.$$

No cross-index interaction occurs during addition. It is a *position-wise* merge.

5.2.2 Visual Walkthrough: 5-D Vector Addition

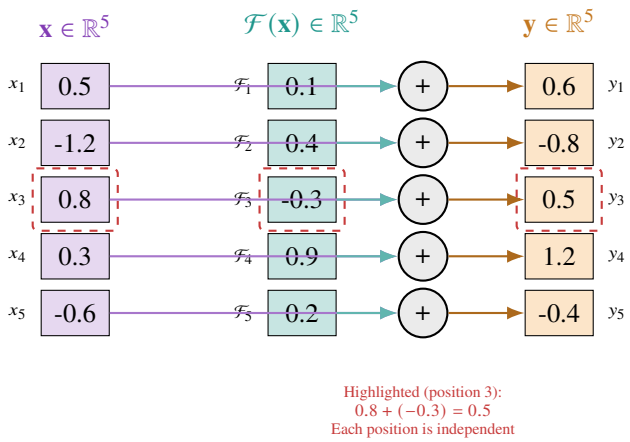


Figure 5.1: Element-wise addition of two 5-D vectors. Each position i contributes an independent scalar addition. The resulting vector $\mathbf{y}$ is fed to ReLU to produce the block output.

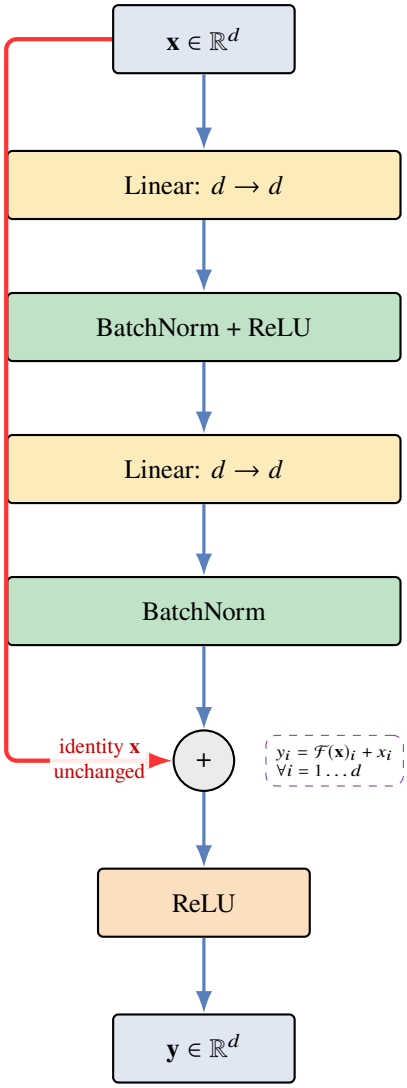


Figure 5.2: FC-style residual block. Both branches must output $\mathbb{R}^d$. The element-wise addition is followed by ReLU to restore nonlinearity.

5.2.3 Full FC Residual Block

5.3 Element-Wise Addition of Feature Maps (CNN Residuals)

5.3.1 Feature Maps as 3-D Tensors

In a convolutional ResNet, data flows as 3-D tensors:

$$X \in \mathbb{R}^{C \times H \times W}, \quad \mathcal{F}(X) \in \mathbb{R}^{C \times H \times W}.$$

The residual sum is:

$$Y_{c,h,w} = X_{c,h,w} + \mathcal{F}(X)_{c,h,w}, \quad c = 1 \ldots C,\ h = 1 \ldots H,\ w = 1 \ldots W.$$

This is $C \times H \times W$ independent scalar additions in parallel—one per spatial location per channel. No cross-channel or cross-spatial mixing occurs *during* the addition.

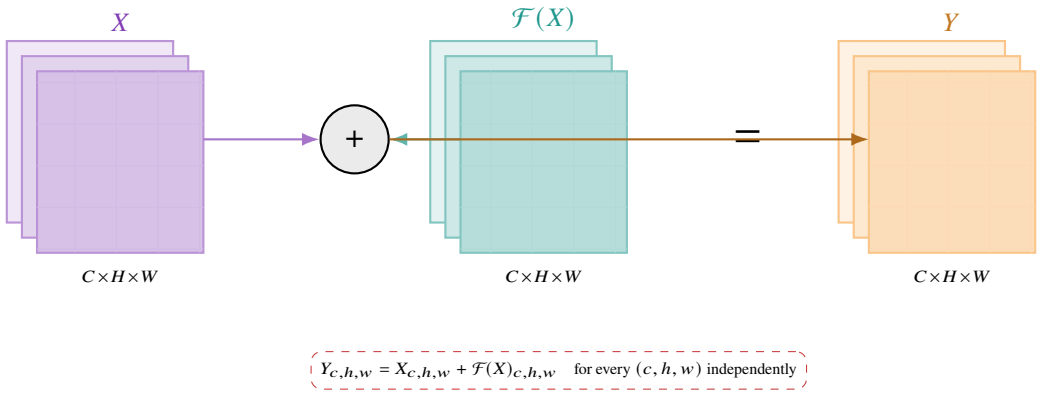


Figure 5.3: Element-wise addition of two feature map tensors. All three dimensions (channel C, height H, width W) must match exactly. The addition is applied cell-by-cell across the entire volume.

5.3.2 Visual Walkthrough: Feature Map Addition

5.4 Identity vs Projection Shortcut

5.5 The Role of ReLU After Addition

The raw sum $Z = \mathcal{F}(X) + X$ is unbounded (can be any real value). Applying ReLU element-wise:

$$Y_{c,h,w} = \max(0,\ Z_{c,h,w})$$

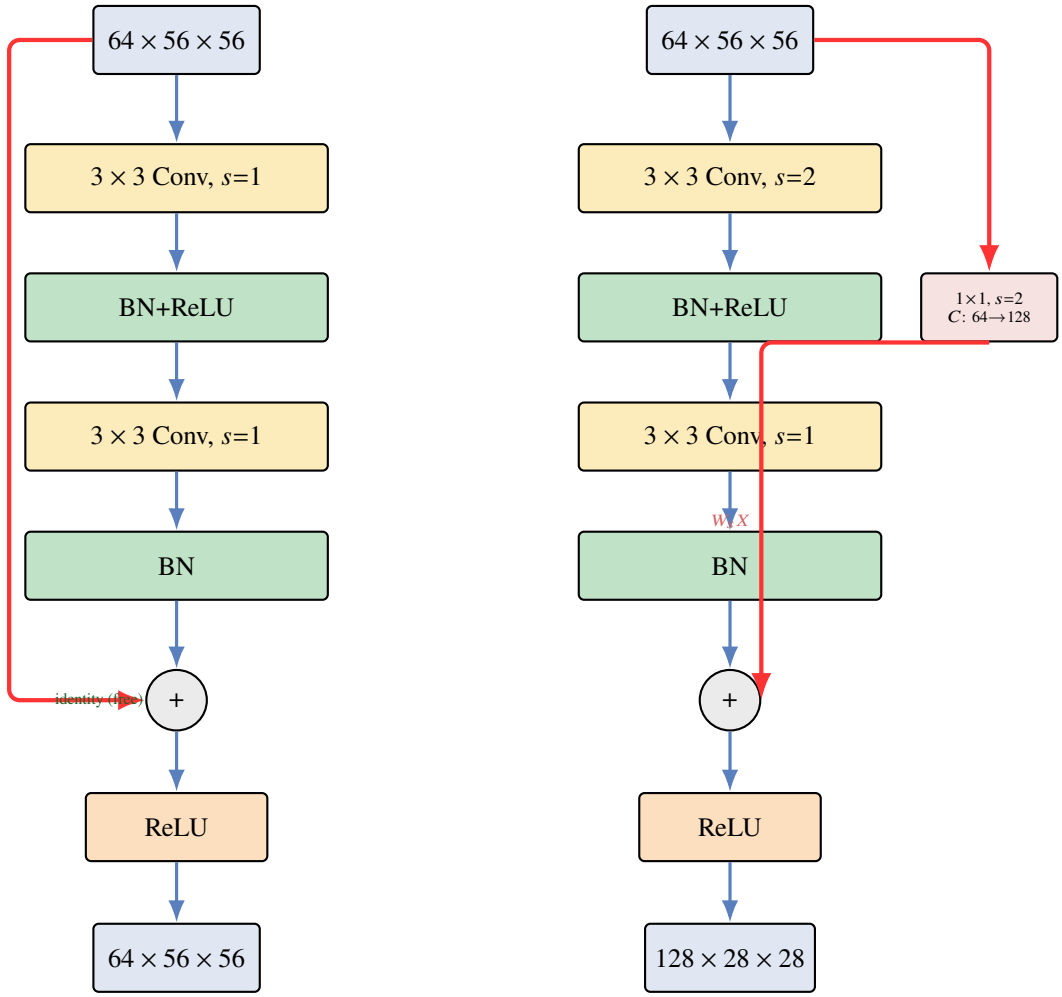
serves two essential roles:

1. **Nonlinear capacity:** Without it, stacking residual blocks collapses to a linear function of the input. ReLU reintroduces the expressiveness necessary for complex tasks.
2. **Sparse activation:** Negative sums become zero, inducing sparsity and implicit regularization. Only informative feature locations remain active.

5.6 Summary: Vector vs Feature-Map Addition

Aspect	Vector (FC/MLP)	Feature map (CNN)
Shape	$\mathbf{x} \in \mathbb{R}^d$	$X \in \mathbb{R}^{C \times H \times W}$
Formula	$y_i = x_i + \mathcal{F}(\mathbf{x})_i$	$Y_{c,h,w} = X_{c,h,w} + \mathcal{F}(X)_{c,h,w}$
Constraint	Both must be $\mathbb{R}^d$	C, H, W must all match
Mismatch fix	Linear proj. $W_s\mathbf{x}$	1×1 conv, stride s
After addition	ReLU	ReLU (or BN→ReLU pre-act)
Used in	MLP-Mixer, Transformer FFN	ResNet, YOLO, U-Net

Table 5.1: Comparison of element-wise addition in vector and feature-map contexts.



(a) Identity Shortcut
same $C, H, W \rightarrow$ free pass

(b) Projection Shortcut
stride-2 doubles channels

Figure 5.4: When spatial size or channel count changes, a 1×1 projection conv aligns the skip path to match $\mathcal{F}(X)$, restoring the ability to add element-wise.

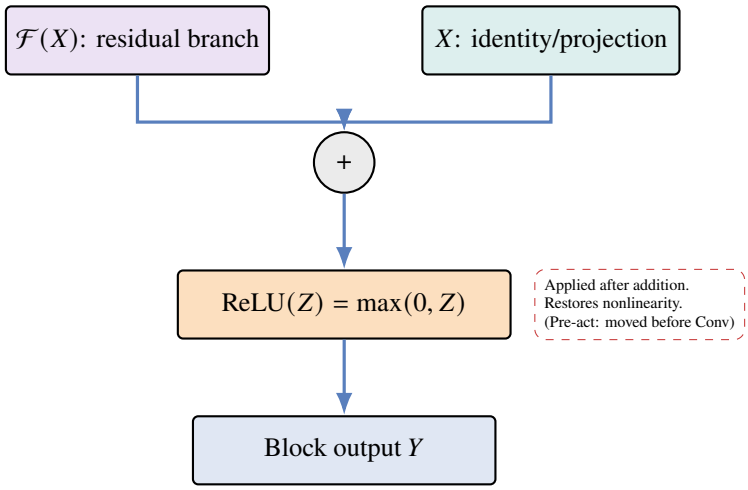


Figure 5.5: Residual and skip branches merge via element-wise addition; ReLU is then applied to the sum.

Chapter 6

Weight Initialization and Normalization

6.1 Kaiming / He Initialization

6.1.1 The Problem with Naive Initialization

If weights are initialized from a standard normal $\mathcal{N}(0, 1)$, the variance of activations grows (or shrinks) exponentially with depth. Consider a linear layer $y = Wx$ with n inputs:

$$\mathrm{Var}(y_i) = n \cdot \mathrm{Var}(W_{ij}) \cdot \mathrm{Var}(x_j).$$

To keep $\mathrm{Var}(y) = \mathrm{Var}(x)$ we need $\mathrm{Var}(W) = 1/n$.

6.1.2 He Initialization (for ReLU)

ReLU zeros out half of its inputs. Accounting for this, He et al. (2015) derived:

$$W \sim \mathcal{N}\left(0, \frac{2}{n_{\text{in}}}\right),$$

where $n_{\text{in}} = C_{\text{in}} \times k \times k$ for a conv layer with C_{in} input channels and kernel size k.

Derivation Sketch

Let x pass through ReLU. Since ReLU kills $\approx 50\%$ of neurons:

$$\mathbb{E}[x_i^2] = \frac{1}{2}\mathrm{Var}(x).$$

For $y = Wx$, we need $\mathrm{Var}(y) = \mathrm{Var}(x)$, giving $n_{\text{in}} \cdot \mathrm{Var}(W) \cdot \frac{1}{2} = 1$, hence $\mathrm{Var}(W) = \frac{2}{n_{\text{in}}}$. □

6.1.3 Zero-Init of Final BN Scale

An additional trick ("ReZero" / "FixUp"): set the final BatchNorm $\gamma = 0$ at initialization, making each residual block initially an identity mapping. This means the full network starts as a pure identity function, and layers gradually "switch on" as training progresses, improving early stability.

$$y = x + \underbrace{\gamma}_{\to 0} \cdot \mathcal{F}(x) \xrightarrow{\gamma=0} y = x.$$

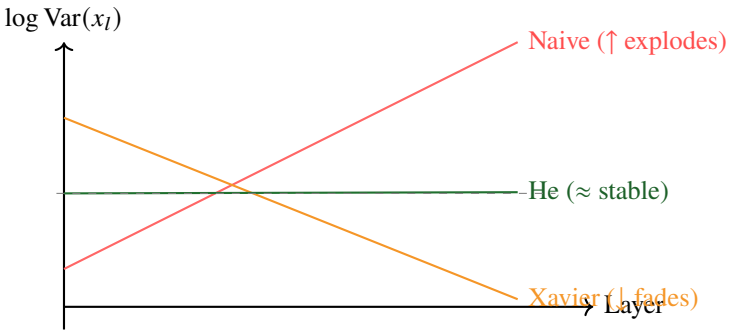


Figure 6.1: Activation variance across layers under different initialization schemes. He initialization keeps variance stable throughout depth, enabling clean forward and backward passes.

6.2 Batch Normalization

6.2.1 Motivation

As the network trains, the distribution of activations in each layer shifts ("internal covariate shift"). Later layers must constantly re-adapt. BatchNorm normalizes each mini-batch to zero mean and unit variance, then rescales with learnable γ, β:

$$\hat{x} = \frac{x - \mu_B}{\sqrt{\sigma_B^2 + \epsilon}}, \qquad y = \gamma \hat{x} + \beta,$$

where μ_B, σ_B^2 are the mini-batch mean and variance.

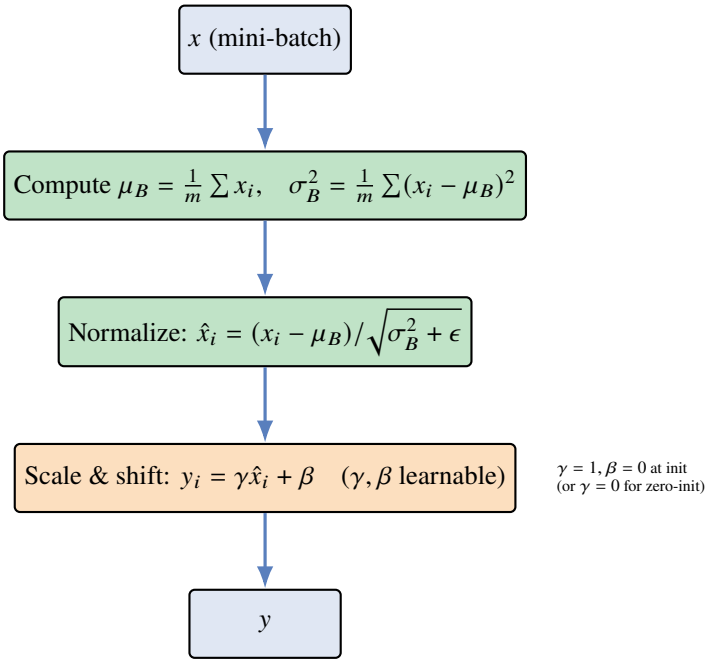


Figure 6.2: BatchNorm computation flow. During inference, running statistics (exponential moving averages computed during training) replace the mini-batch statistics.

6.2.2 BN Position: Before or After Addition?

6.2.3 Alternatives to BatchNorm

- **GroupNorm:** Normalizes within groups of channels. Stable for batch size 1.

	Post-activation (v1)	**Pre-activation (v2)**
BN position	After conv, before skip add	Before conv, on main path only
Skip path	Passes through final ReLU	Pure identity (no distortion)
Gradient path	Slightly distorted	Perfectly clean
Best for	ResNet-18/34/50	ResNet-1001+

- **LayerNorm:** Normalizes across all channels/spatial positions. Standard in Transformers.
- **InstanceNorm:** Normalizes per sample per channel. Used in style transfer.

Chapter 7

Training ResNets: Schedules, Regularization, and Tricks

7.1 Learning Rate Schedules

7.1.1 Step Decay

The simplest schedule: multiply η by factor γ every k epochs.

$$\eta_t = \eta_0 \cdot \gamma^{\lfloor t/k \rfloor}.$$

Common setting: $\gamma = 0.1$, $k = 30$ epochs.

7.1.2 Cosine Annealing

Smoothly decays the learning rate following a cosine curve, allowing the optimizer to escape sharp minima late in training:

$$\eta_t = \eta_{\min} + \frac{\eta_{\max} - \eta_{\min}}{2}\left(1 + \cos\frac{t\pi}{T}\right).$$

7.1.3 Linear Warmup

For large-batch training (> 512 samples), the gradient noise is low and a high initial LR can destabilize early training. A linear warmup ramps from $\eta_{\min}$ to $\eta_{\max}$ over w epochs before decaying:

$$\eta_t = \eta_{\max} \cdot \frac{t}{w}, \quad t < w.$$

7.2 Stochastic Depth (DropPath)

Huang et al. (2016) proposed randomly *dropping entire residual blocks* during training with probability p_l, linearly increasing with depth:

$$p_l = \frac{l}{L} \cdot p_L, \quad \tilde{y} = \begin{cases} y & \text{with prob. } 1 - p_l \\ x & \text{(bypass)} \end{cases}.$$

Benefits:

- Acts like Dropout but at the block level.
- Reduces effective depth during training (faster).
- Ensemble interpretation: each training step samples a different sub-network.

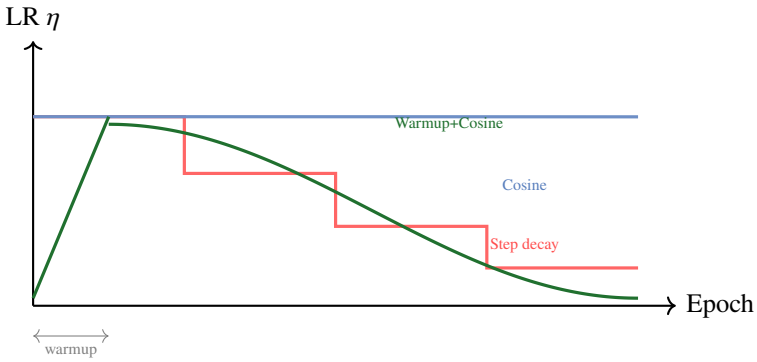


Figure 7.1: Three common learning rate schedules. Warmup+Cosine is now the default for ResNet training on ImageNet and is required when using large batches (>256).

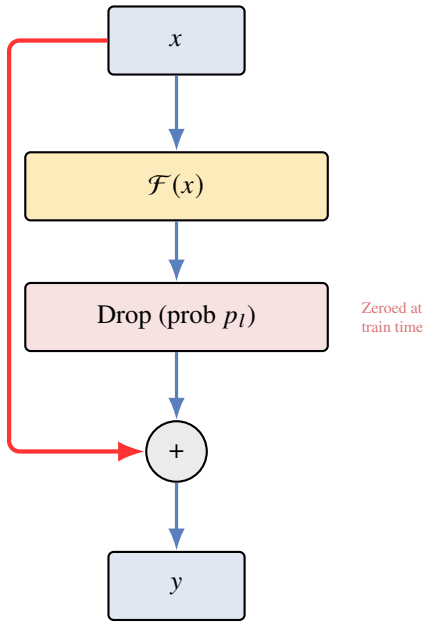


Figure 7.2: Stochastic Depth: the residual branch is zeroed with probability p_l, effectively bypassing the block. At test time, the full network is used (with scaled outputs).

7.3 Data Augmentation Strategies

Method	Description	Top-1 gain
Random Crop	Crop 224×224 from random location of resized image	+1.0%
Random Flip	Horizontal flip with $p = 0.5$	+0.5%
Color Jitter	Random brightness, contrast, saturation, hue	+0.3%
Mixup	Blend two images: $\tilde{x} = \lambda x_i + (1 - \lambda) x_j$	+0.5%
CutMix	Paste rectangular patch from one image into another	+0.6%
RandAugment	Apply 2 random augmentations from a 14-op pool	+0.7%
Label Smoothing	Replace one-hot labels with $[0.9, 0.1/(K-1)]$	+0.3%

Table 7.1: Common augmentation and regularization strategies for ResNet training on ImageNet.

7.4 Complete Training Recipe (ResNet-50 on ImageNet)

ResNet-50 Training Configuration

```
config = {
    # Data
    "dataset"         : "ImageNet-1K",
    "input_size"      : 224,
    "batch_size"      : 256,          # per GPU
    "num_workers"     : 8,

    # Optimizer
    "optimizer"       : "SGD",
    "momentum"        : 0.9,
    "weight_decay"    : 1e-4,

    # Schedule
    "epochs"          : 90,
    "lr_max"          : 0.1,          # scaled: 0.1 * batch/256
    "warmup_epochs"   : 5,
    "schedule"        : "cosine",

    # Regularization
    "label_smoothing": 0.1,
    "mixup_alpha"     : 0.2,
    "drop_path_rate" : 0.1,

    # Initialization
    "init"            : "kaiming_normal",
    "bn_final_gamma" : 0,             # zero-init last BN
}
```

Chapter 8

PyTorch Implementation

8.1 Basic Residual Block

BasicBlock — ResNet-18/34

```python
import torch
import torch.nn as nn
import torch.nn.functional as F

class BasicBlock(nn.Module):
    expansion = 1  # output channels = planes * expansion

    def __init__(self, in_planes, planes, stride=1):
        super().__init__()
        # Main path: two 3x3 convolutions
        self.conv1 = nn.Conv2d(in_planes, planes, 3,
                               stride=stride, padding=1, bias=False)
        self.bn1   = nn.BatchNorm2d(planes)

        self.conv2 = nn.Conv2d(planes, planes, 3,
                               stride=1, padding=1, bias=False)
        self.bn2   = nn.BatchNorm2d(planes)
        # Zero-init final BN (identity at start)
        nn.init.constant_(self.bn2.weight, 0)

        # Projection shortcut (when shapes differ)
        self.shortcut = nn.Sequential()
        if stride != 1 or in_planes != planes:
            self.shortcut = nn.Sequential(
                nn.Conv2d(in_planes, planes, 1,
                          stride=stride, bias=False),
                nn.BatchNorm2d(planes)
            )

    def forward(self, x):
        # Residual branch
        out = F.relu(self.bn1(self.conv1(x)))
        out = self.bn2(self.conv2(out))
        # Element-wise addition (MUST have same shape!)
        out = out + self.shortcut(x)  # <-- the core operation
        out = F.relu(out)             # <-- activation AFTER addition
        return out
```

8.2 Bottleneck Block

Bottleneck — ResNet-50/101/152

```python
class Bottleneck(nn.Module):
    expansion = 4  # output channels = planes * 4

    def __init__(self, in_planes, planes, stride=1):
        super().__init__()
        # 1x1 reduce
        self.conv1 = nn.Conv2d(in_planes, planes, 1, bias=False)
        self.bn1   = nn.BatchNorm2d(planes)
        # 3x3 process
        self.conv2 = nn.Conv2d(planes, planes, 3,
                               stride=stride, padding=1, bias=False)
        self.bn2   = nn.BatchNorm2d(planes)
        # 1x1 expand
        self.conv3 = nn.Conv2d(planes, planes * self.expansion,
                               1, bias=False)
        self.bn3   = nn.BatchNorm2d(planes * self.expansion)
        nn.init.constant_(self.bn3.weight, 0)  # zero-init

        # Projection shortcut
        self.shortcut = nn.Sequential()
        out_planes = planes * self.expansion
        if stride != 1 or in_planes != out_planes:
            self.shortcut = nn.Sequential(
                nn.Conv2d(in_planes, out_planes, 1,
                          stride=stride, bias=False),
                nn.BatchNorm2d(out_planes)
            )

    def forward(self, x):
        out = F.relu(self.bn1(self.conv1(x)))    # reduce
        out = F.relu(self.bn2(self.conv2(out)))  # process
        out = self.bn3(self.conv3(out))           # expand
        out = out + self.shortcut(x)             # element-wise add
        out = F.relu(out)
        return out
```

8.3 Full ResNet Class

ResNet — Configurable depth

```python
class ResNet(nn.Module):
    def __init__(self, block, num_blocks, num_classes=1000):
        super().__init__()
        self.in_planes = 64

        # Stem: 7x7 conv + maxpool
        self.stem = nn.Sequential(
            nn.Conv2d(3, 64, 7, stride=2, padding=3, bias=False),
            nn.BatchNorm2d(64),
            nn.ReLU(inplace=True),
            nn.MaxPool2d(3, stride=2, padding=1)
```

```python
        )
        # Residual stages
        self.stage1 = self._make_stage(block, 64,  num_blocks[0], stride=1)
        self.stage2 = self._make_stage(block, 128, num_blocks[1], stride=2)
        self.stage3 = self._make_stage(block, 256, num_blocks[2], stride=2)
        self.stage4 = self._make_stage(block, 512, num_blocks[3], stride=2)
        # Head
        self.avgpool = nn.AdaptiveAvgPool2d((1, 1))  # Global Average Pool
        self.fc = nn.Linear(512 * block.expansion, num_classes)

        # He initialization
        for m in self.modules():
            if isinstance(m, nn.Conv2d):
                nn.init.kaiming_normal_(m.weight, mode='fan_out',
                                        nonlinearity='relu')

    def _make_stage(self, block, planes, num_blocks, stride):
        layers = [block(self.in_planes, planes, stride)]
        self.in_planes = planes * block.expansion
        for _ in range(1, num_blocks):
            layers.append(block(self.in_planes, planes, stride=1))
        return nn.Sequential(*layers)

    def forward(self, x):
        x = self.stem(x)           # 3x224x224 -> 64x56x56
        x = self.stage1(x)         # 64x56x56
        x = self.stage2(x)         # 128x28x28
        x = self.stage3(x)         # 256x14x14
        x = self.stage4(x)         # 512x7x7
        x = self.avgpool(x)        # 512x1x1
        x = torch.flatten(x, 1)   # 512
        x = self.fc(x)             # num_classes
        return x

# Instantiate standard ResNets
def ResNet18():  return ResNet(BasicBlock,   [2,2,2,2])
def ResNet34():  return ResNet(BasicBlock,   [3,4,6,3])
def ResNet50():  return ResNet(Bottleneck,   [3,4,6,3])
def ResNet101(): return ResNet(Bottleneck,   [3,4,23,3])
def ResNet152(): return ResNet(Bottleneck,   [3,8,36,3])
```

8.4 Transfer Learning with ResNet

Fine-tuning a pretrained ResNet-50

```python
import torchvision.models as models

# Load pretrained backbone
model = models.resnet50(weights='IMAGENET1K_V2')
```

```python

# Freeze all layers
for param in model.parameters():
    param.requires_grad = False

# Replace final classifier for new task (e.g., 10 classes)
model.fc = nn.Linear(model.fc.in_features, 10)
# Only fc parameters will be updated
optimizer = torch.optim.Adam(model.fc.parameters(), lr=1e-3)

# --- Phase 2: Unfreeze and fine-tune deeper layers ---
for param in model.layer4.parameters():
    param.requires_grad = True
optimizer2 = torch.optim.SGD([
    {'params': model.layer4.parameters(), 'lr': 1e-4},
    {'params': model.fc.parameters(),     'lr': 1e-3}
], momentum=0.9, weight_decay=1e-4)
```

8.5 Common Failure Modes and Debugging

Symptom	Likely Cause	Fix
Loss NaN at start	LR too high, no warmup	Add warmup; lower initial LR
Training loss flat	LR too low, or bad init	Increase LR; check init
Shape mismatch error	Skip-path channels mismatch	Add/fix projection shortcut
Overfitting on small data	Too deep, no regularization	Stochastic depth, weight decay, mixup
Slow convergence	No BN momentum schedule	Use cosine annealing
High GPU memory	Large batch on deep model	Gradient checkpointing; reduce batch
BN stats wrong at test	Frozen BN in fine-tuning	Set `model.train()` or freeze BN properly

Table 8.1: Common ResNet training issues and their solutions.

Chapter 9

Theoretical Foundations

9.1 ResNets as Discretized ODEs

Each residual block computes:

$$x_{l+1} = x_l + \mathcal{F}_l(x_l).$$

Viewing l as discrete time with step $\Delta t = 1$:

$$\frac{x_{l+1} - x_l}{\Delta t} = \mathcal{F}_l(x_l) \implies \frac{dx}{dt} = f(x(t), t).$$

This is the **forward Euler discretization** of a continuous ODE.

9.1.1 Neural ODEs (Chen et al., 2018)

Taking $\Delta t \to 0$ yields a *Neural ODE*: a network defined by an ODE solved by a black-box integrator. Backpropagation is replaced by the **adjoint sensitivity method**:

$$\frac{d\mathbf{a}}{dt} = -\mathbf{a}(t)^\top \frac{\partial f}{\partial x}, \quad \mathbf{a}(t) = \frac{\partial \mathcal{L}}{\partial x(t)}.$$

This enables $O(1)$ memory backpropagation regardless of depth.

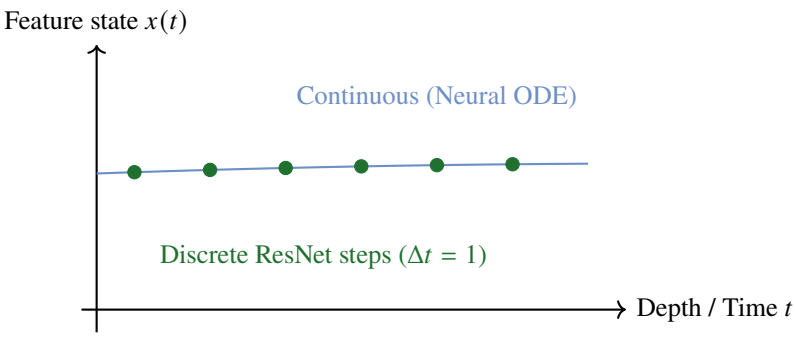


Figure 9.1: ResNet layers as Euler steps sampling an underlying continuous feature trajectory.

9.2 Ensemble Interpretation

In a ResNet with L blocks, the output can be written as a sum over 2^L paths:

$$x_L = \sum_{S \subseteq \{1,\ldots,L\}} \left(\prod_{l \in S} \mathcal{F}_l \right)(x_0).$$

At test time, *all* paths are active. Removing one block changes only 2^{L-1} of these paths—a graceful degradation rather than catastrophic failure. This explains the observed robustness to layer removal in ResNets.

9.3 Loss Landscape Analysis

Li et al. (2018) visualized ResNet loss surfaces by projecting the high-dimensional parameter space onto 2D planes. Key findings:

- ResNets produce **convex-like, smooth** loss surfaces.
- Plain CNNs of comparable depth produce **chaotic, multi-peaked** surfaces.
- Width and skip connections are the primary contributors to smoothness.

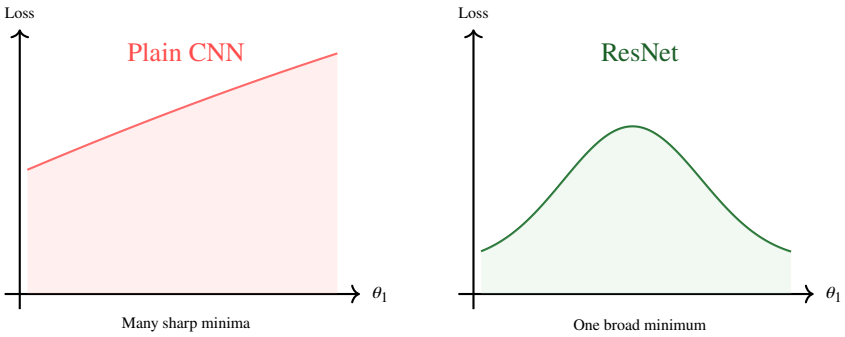


Figure 9.2: 1D cross-sections of loss landscapes. ResNets produce wide, smooth basins that are easier for SGD to find and converge into.

9.4 Gradient Flow Analysis

The full gradient equation for L residual blocks:

$$\frac{\partial \mathcal{L}}{\partial x_0} = \frac{\partial \mathcal{L}}{\partial x_L} \underbrace{\prod_{l=0}^{L-1} \left(I + \frac{\partial \mathcal{F}_l}{\partial x_l} \right)}_{\text{never vanishes (identity term)}} .$$

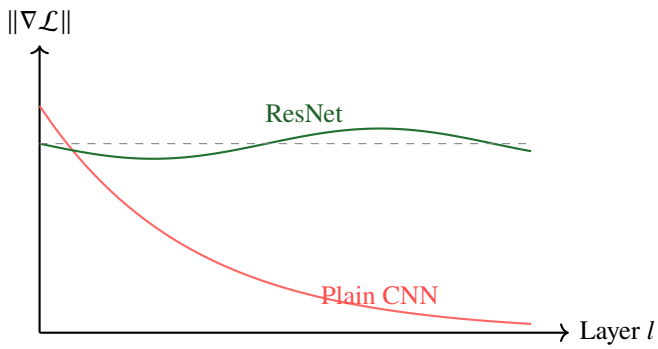


Figure 9.3: Gradient magnitude across layers. ResNets maintain near-constant gradient norms through the identity term in the Jacobian product.

Chapter 10

Modern Architecture Variants

10.1 DenseNet: All-to-All Connections

DenseNet (Huang et al., 2017) generalizes the skip connection idea: *every* layer receives concatenated feature maps from all previous layers:

$$x_l = \mathcal{H}_l([x_0, x_1, \ldots, x_{l-1}]).$$

Instead of element-wise addition, DenseNet uses **channel concatenation**, preserving all features explicitly and reusing them at every subsequent layer.

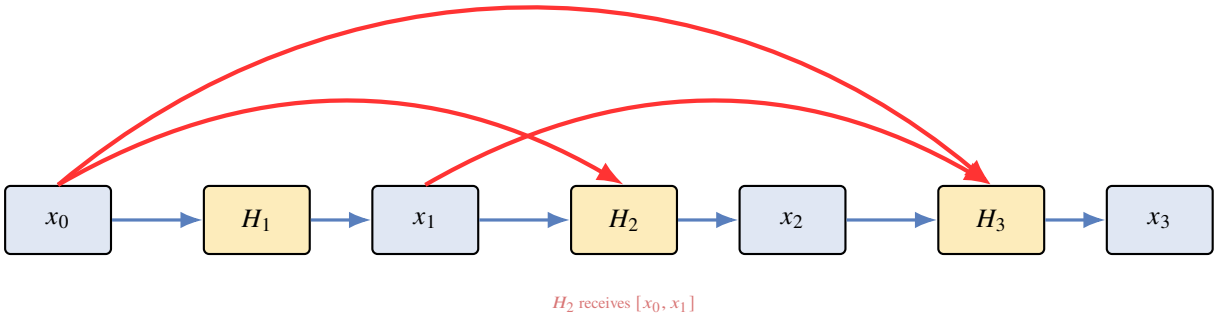


Figure 10.1: DenseNet connectivity. Every layer receives feature maps from all preceding layers via concatenation. This maximizes feature reuse but increases memory usage as depth grows.

10.2 ConvNeXt: Modernizing ResNets (2022)

Liu et al. (2022) applied Transformer design choices to a ResNet backbone:

10.3 Vision Transformers (ViT) and Residuals

Despite replacing convolutions with self-attention, ViTs retain the residual structure:

$$x_{l+1} = x_l + \text{Attention}(x_l), \qquad x_{l+2} = x_{l+1} + \text{MLP}(x_{l+1}).$$

Every Transformer block is itself a pair of residual connections—a direct inheritance from ResNets.

10.4 Architecture Family Comparison

Change	**ResNet-50**	**ConvNeXt-T**
Stem	7×7, stride 4	4×4, stride 4 (patchify)
Depthwise conv	None	7×7 depthwise conv
Normalization	BatchNorm	LayerNorm
Activation	ReLU	GELU
Inverted bottleneck	No	Yes (4× expansion)
Stage ratio	3-4-6-3	3-3-9-3
Top-1 Acc	76.1%	82.1%

Table 10.1: ConvNeXt modernizes ResNet design choices using lessons from Vision Transformers, gaining 6% accuracy with similar FLOPs.

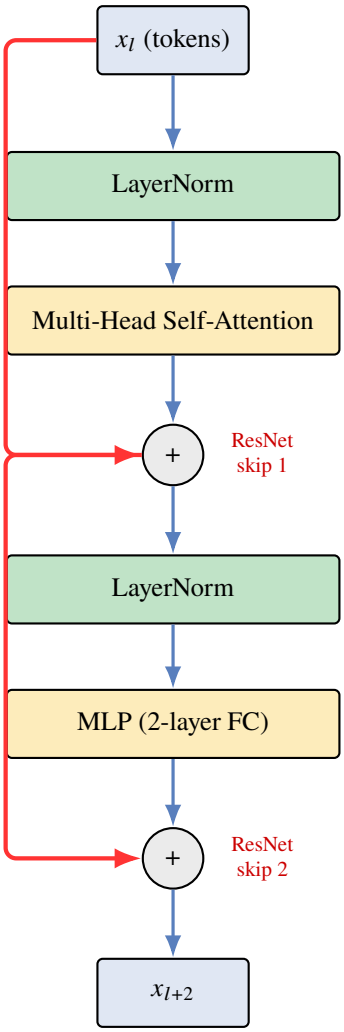


Figure 10.2: A Vision Transformer block. Both the attention and MLP sub-blocks use pre-activation residual connections identical to He et al. (2016). ViTs are ResNets with attention replacing convolution.

Model	**Skip type**	**Params**	**Top-1**	**Key innovation**
ResNet-50	Add	25.6M	76.1%	Identity shortcuts
ResNet-101	Add	44.5M	77.4%	Deeper bottleneck
WRN-28-10	Add	36.5M	78.9%	Wider channels
ResNeXt-50	Add	25.0M	77.8%	Grouped convs
SE-ResNet-50	Add+scale	28.1M	77.6%	Channel attention
DenseNet-121	Concat	8.0M	74.4%	Dense reuse
ConvNeXt-T	Add	28.6M	82.1%	Modern design
ViT-B/16	Add	86.6M	81.8%	Self-attention

Table 10.2: Comparison of ResNet-inspired architectures on ImageNet-1K.

Chapter 11

Applications and Transfer Learning

11.1 Computer Vision Tasks

11.1.1 Image Classification

The canonical task. ResNet-50 pretrained on ImageNet achieves >90% top-5 on most domain-specific datasets after fine-tuning with only the classifier replaced.

11.1.2 Object Detection: Faster R-CNN and YOLO

ResNets serve as the **backbone** (feature extractor) in two-stage and one-stage detectors:

- **Faster R-CNN + ResNet-101:** 36.8 mAP on COCO 2017.
- **YOLOv3 + Darknet-53 (ResNet-style):** Real-time detection at 30 FPS.
- The Feature Pyramid Network (FPN) taps into multiple ResNet stages simultaneously for multi-scale detection.

11.1.3 Semantic Segmentation: DeepLab and U-Net

- **DeepLabv3+:** ResNet-101 backbone with atrous convolutions and ASPP module. 82.1% mIoU on Pascal VOC.
- **U-Net:** Skip connections between encoder and decoder stages carry high-resolution features to the upsampling path (a form of feature-level skip, not residual addition).

11.2 Medical Imaging

ResNets are the dominant backbone in medical AI:

Application	Dataset	Performance
Diabetic retinopathy	EyePACS (88K images)	AUC 0.99 (ResNet-50 + ensemble)
Chest X-ray	CheXNet (112K X-rays)	Exceeds radiologist on 14 pathologies
Skin cancer	ISIC (25K images)	AUC 0.96 (vs 0.87 dermatologist)
MRI segmentation	BraTS (brain tumors)	Dice 0.89 (ResU-Net)

Table 11.1: Selected medical imaging benchmarks using ResNet backbones.

11.3 Beyond Vision

- **Audio:** Spectrogram images fed to ResNet-34 achieve state-of-the-art on AudioSet classification.
- **NLP:** Transformer FFN sublayers are residual MLP blocks. BERT, GPT, T5 all inherit residual structure.
- **Graphs:** Graph ResNets add node feature vectors element-wise across GNN layers, preventing over-smoothing.
- **Reinforcement Learning:** Impala (Espeholt et al., 2018) uses ResNet-style convolutional backbones for visual policy learning in Atari and DM-Lab.
- **Protein Structure:** AlphaFold2 uses residual Evoformer blocks for protein structure prediction.

Chapter 12

Limitations and Open Research Directions

12.1 Known Limitations

Limitation	Details
Linear compute cost	FLOPs scale linearly with depth; doubling layers doubles compute
Memory overhead	All intermediate activations needed for backprop; mitigated by gradient checkpointing
Shape constraint	Element-wise addition requires matching C, H, W; projections add parameters
Diminishing returns	Accuracy gains taper beyond ~150 layers on standard benchmarks
Inductive bias	Local receptive fields limit global context; ViTs address this
BN instability	BatchNorm fails with batch size 1; requires GroupNorm/LayerNorm alternatives

Table 12.1: Fundamental limitations of the ResNet architecture.

12.2 Active Research Directions

12.2.1 Dynamic Residual Paths

Adaptive Computation Time and **Early Exit** networks learn to skip residual blocks at inference based on input difficulty, reducing average compute while maintaining accuracy.

12.2.2 Neural Architecture Search (NAS)

AutoML approaches search the space of block types, skip patterns, and channel widths automatically. EfficientNet (Tan & Le, 2019) discovered compound scaling laws relating depth, width, and resolution.

12.2.3 Zero-Cost Proxies

Research into predicting which ResNet architectures will perform well *without* training, using gradient flow statistics and feature map statistics computed in a single forward pass.

12.2.4 Mechanistic Interpretability

Recent work analyzes what individual residual blocks compute, finding that early blocks detect textures, middle blocks detect shapes, and late blocks detect semantic categories—consistent with the ensemble

view.

References

1. K. He, X. Zhang, S. Ren, J. Sun. *Deep Residual Learning for Image Recognition*. CVPR, 2016.
2. K. He, X. Zhang, S. Ren, J. Sun. *Identity Mappings in Deep Residual Networks*. ECCV, 2016.
3. S. Ioffe, C. Szegedy. *Batch Normalization: Accelerating Deep Network Training by Reducing Internal Covariate Shift*. ICML, 2015.
4. K. Simonyan, A. Zisserman. *Very Deep Convolutional Networks for Large-Scale Image Recognition*. ICLR, 2015.
5. S. Zagoruyko, N. Komodakis. *Wide Residual Networks*. BMVC, 2016.
6. S. Xie, R. Girshick, P. Dollar, Z. Tu, K. He. *Aggregated Residual Transformations for Deep Neural Networks*. CVPR, 2017.
7. J. Hu, L. Shen, G. Sun. *Squeeze-and-Excitation Networks*. CVPR, 2018.
8. G. Huang, Z. Liu, L. van der Maaten, K. Weinberger. *Densely Connected Convolutional Networks*. CVPR, 2017.
9. R. T. Chen, Y. Rubanova, J. Bettencourt, D. Duvenaud. *Neural Ordinary Differential Equations*. NeurIPS, 2018.
10. H. Li, Z. Xu, G. Taylor, C. Studer, T. Goldstein. *Visualizing the Loss Landscape of Neural Nets*. NeurIPS, 2018.
11. A. Veit, M. Wilber, S. Belongie. *Residual Networks Behave Like Ensembles of Relatively Shallow Networks*. NeurIPS, 2016.
12. G. Huang, Y. Sun, Z. Liu, D. Sedra, K. Weinberger. *Deep Networks with Stochastic Depth*. ECCV, 2016.
13. T. He, Z. Zhang, H. Zhang, Z. Zhang, J. Xie, M. Li. *Bag of Tricks for Image Classification with Convolutional Neural Networks*. CVPR, 2019.
14. K. Liu, H. Mao, C.-Y. Wu, C. Feichtenhofer, T. Darrell, S. Xie. *A ConvNet for the 2020s (ConvNeXt)*. CVPR, 2022.
15. A. Dosovitskiy et al. *An Image is Worth 16×16 Words: Transformers for Image Recognition at Scale*. ICLR, 2021.
16. M. Tan, Q. V. Le. *EfficientNet: Rethinking Model Scaling for Convolutional Neural Networks*. ICML, 2019.
17. K. He, X. Zhang, S. Ren, J. Sun. *Delving Deep into Rectifiers: Surpassing Human-Level Performance on ImageNet Classification*. ICCV, 2015.

18. S. Ren, K. He, R. Girshick, J. Sun. *Faster R-CNN: Towards Real-Time Object Detection with Region Proposal Networks*. NeurIPS, 2015.

19. J. Redmon, A. Farhadi. *YOLOv3: An Incremental Improvement*. arXiv, 2018.

20. L.-C. Chen, Y. Zhu, G. Papandreou, F. Schroff, H. Adam. *Encoder-Decoder with Atrous Separable Convolution for Semantic Segmentation (DeepLabv3+)*. ECCV, 2018.